DATA ANALYST INTERNSHIP

Advanced Excel Functions and Automation

An annual nutritional-supply early-warning report for NutriLine Foods, built on FAO data (Project FOODFLOW)

Prepared by: Ayush Yadav
Date: 17 October 2026
Workbook: Food_supply_FAO_2020.xlsm (Excel with a VBA macro)

5	1	8	0
Advanced formula techniques	VBA macro: GenerateActionReport	Countries flagged as Severe Deficit in 2017	Mismatches in 11,028 rows checked independently

About the data

The data is real: FAO food supply per person for 216 countries, regions and groups, 1961 to 2017, the same dataset used in Week 2. The **nutritional thresholds** (2,100 and 2,500 kcal a day) are screening rules chosen for this exercise. They are not official classifications.

1. Introduction and Objective

1.1 Context

Week 1 planned Project FOODFLOW and Week 2 cleaned and analysed FAO food supply data with PivotTables. Week 3 builds on the same workbook and moves from analysing data to **automating a routine task**: every year, someone must find the countries where food supply per person is dangerously low. Done by hand, that means filtering an 11,028-row table three different ways and copying the results.

1.2 Objectives

- **Define a scenario** in the virtual food processing sector that benefits from automation.
- **Use advanced Excel functions** (five techniques here, above the minimum of three) to solve specific data challenges in that scenario.
- **Write a VBA macro** that automates the repetitive part, and document its logic and intended outcome.
- **Provide a tutorial** that lets a reader rebuild and run it, and **validate** the formulas and the macro so the results can be trusted.

2. Scenario Definition

2.1 The scenario: an annual nutritional-supply early-warning list

NutriLine Foods, the virtual plant from Weeks 1 and 2, wants a yearly early-warning list of countries where food energy supply per person is far below what people need. The list helps the company decide where to monitor demand and where shelf-stable, processed products could matter most. Each year the analyst must do the same four things: grade every country, pick out the worst, summarise how bad the shortfall is, and produce a clean list for managers.

2.2 Why it needs automation

Manual task	What can go wrong	Automated with
Tell countries from regions and former states	Regions double-count countries	INDEX/MATCH lookup (Type column)
Grade every row by calories per person	Inconsistent grading, boundary mistakes	Nested IF (Nutritional_Status column)
Average a measure for a filtered group	Wrong filter, hand-copied numbers	SUMPRODUCT with several conditions
Find the highest value and who holds it	Reading the wrong row	INDEX/MATCH with MAX
Copy the flagged countries into a report	Missed rows, copy errors, stale lists	VBA macro: GenerateActionReport

2.3 The grading rule

Status	Calories per person per day	Meaning in this report
Severe Deficit	Below 2,100	Priority flag: food supply is far below need.
Warning	2,100 to below 2,500	Watch list: supply is thin.
Adequate	2,500 or more	No flag.

These thresholds are screening rules chosen for this exercise. Food supply is also not the same as food eaten, so a flag is a prompt for investigation and not a diagnosis (see Section 9).

3. Workbook Design

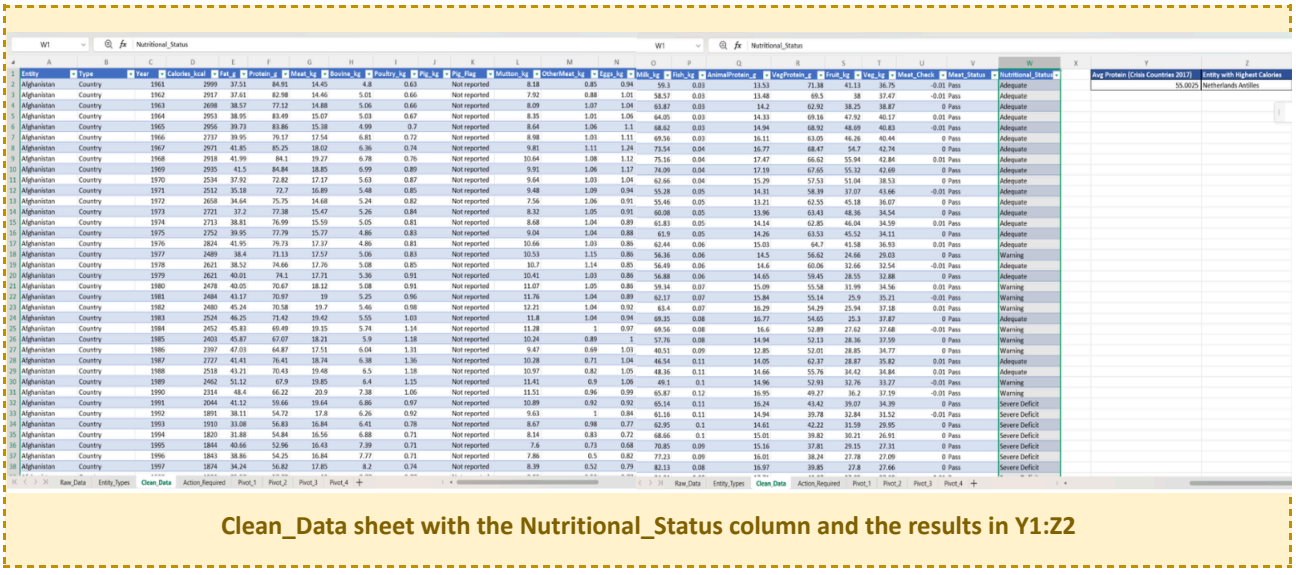
Week 3 extends the Week 2 workbook. Nothing was rebuilt: the new work sits beside the existing sheets.

Sheet or item	Contents
Raw_Data	The original FAO file, never edited (Week 2).
Entity_Types	Lookup list of 216 entities, each marked Country, Aggregate, or Historical (Week 2).
Clean_Data	The table tblFood: 11,028 rows and 23 columns. Week 3 adds the column **Nutritional_Status** (column W) and two result cells beside the table (Y2 and Z2).

Sheet or item	Contents
Action_Required	Created by the macro: the list of 2017 Severe Deficit countries.
Pivot_1 to Pivot_4	The Week 2 PivotTables, unchanged.
Module1 (VBA)	The macro GenerateActionReport.

3.1 What Week 3 added

Item	Location	Purpose
Nutritional_Status	Clean_Data!W	Grades every row as Severe Deficit, Warning, or Adequate (nested IF).
Avg Protein (Crisis Countries 2017)	Clean_Data!Y1: Y2	Average protein supply of the 2017 Severe Deficit countries (SUMPRODUCT).
Entity with Highest Calories	Clean_Data!Z1: Z2	Looks up which entity has the highest calorie figure (INDEX/MATCH with MAX).
GenerateActionReport	Module1	Builds the Action_Required report in one click.
Action_Required	New sheet	The macro's output: eight countries.



Clean_Data sheet with the Nutritional_Status column and the results in Y1:Z2

Figure 1. The Week 3 additions on the Clean_Data sheet.

4. Advanced Excel Functions

Each technique follows the same pattern: the problem, the formula, how it works, and a worked example from the workbook.

4.1 INDEX/MATCH: telling countries from regions

Problem. The data mixes 8,977 country rows with 1,771 region or group rows (such as World) and 280 rows for former states. Adding them together would count some people twice.

Formula (Clean_Data, Type column):

```
=INDEX(Entity_Types!$B:$B, MATCH(tblFood[[#This Row],[Entity]], Entity_Types!$A:$A, 0))
```

- **MATCH** finds the position of the row's entity in the list of entity names (the 0 means an exact match).
- **INDEX** returns the value from column B (the Type) in that same position.

Worked example. For Afghanistan, MATCH finds the name in row 2 of Entity_Types, and INDEX returns the Type from B2: **Country**. For Netherlands Antilles, MATCH finds row 137, and INDEX returns **Historical**. This one column is what lets every later formula and the macro ask for "Country" only.

4.2 Nested IF: grading every row

Problem. Every row needs the same, consistent grade, including at the exact boundaries.

Formula (Clean_Data, Nutritional_Status column):

```
=IF(D2 < 2100, "Severe Deficit", IF(D2 < 2500, "Warning", "Adequate"))
```

The tests run in order, and the first true one wins. A reading below 2,100 is graded Severe Deficit, and the formula stops. Only readings of 2,100 or more reach the second test. The order matters: if the 2,500 test came first, every severe case would be graded Warning.

Worked examples (real rows):

Row	Calories	Result	Why
Central African Republic, 2017	1,758	**Severe Deficit**	1,758 is below 2,100.
Zimbabwe, 2017	2,173	**Warning**	Not below 2,100, but below 2,500.
Afghanistan, 1961	2,999	**Adequate**	2,999 is 2,500 or more.

Counts. Across all 11,028 rows, the column gives 5,994 Adequate, 2,980 Warning, and 2,054 Severe Deficit. For the 172 countries in 2017, it gives 131 Adequate, 33 Warning, and **8 Severe Deficit**. Note that the grade is also applied to region and historical rows (295 aggregate rows and 63 historical rows are graded Severe Deficit). That is harmless here, but it is exactly why the macro also tests that the Type is "Country".

4.3 SUMPRODUCT: an average with several conditions

Problem. What is the average protein supply of the countries in severe deficit in 2017?

Formula (Clean_Data, cell Y2):

```
=SUMPRODUCT(--(C2:C11029=2017), --(B2:B11029="Country"), --(W2:W11029="Severe Deficit"), F2:F11029)
  / SUMPRODUCT(--(C2:C11029=2017), --(B2:B11029="Country"), --(W2:W11029="Severe Deficit"))
```

- Each test, such as C2:C11029=2017, produces a column of TRUE and FALSE. The double minus signs (--) turn them into 1 and 0.
- SUMPRODUCT multiplies the columns row by row. A row scores 1 only if **all** conditions are true (the equivalent of AND), so it works as a filter.
- The top half adds up the protein of the matching rows. The bottom half counts the matching rows. Their ratio is the average.

Worked example. The eight matching countries have protein supplies of 54.09, 61.47, 46.44, 70.51, 42.94, 52.25, 53.23, and 59.09 g. They sum to 440.02, and 440.02 / 8 = **55.0025 g**, which is the value in Y2. For comparison, the average across all 172 countries in 2017 is **81.3 g**.

4.4 INDEX/MATCH with MAX: finding who holds the highest value

Problem. Which entity has the highest calorie figure?

Formula as built (Clean_Data, cell Z2):

```
=INDEX(A2:A11029, MATCH(MAX(D2:D11029), D2:D11029, 0))
```

MAX finds the largest calorie value in the column (3,877). MATCH finds the first row that holds it. INDEX returns the entity name from column A in that row. The cell shows **Netherlands Antilles**.

A scope finding. The formula searches every row, so it answers "the highest value anywhere in the dataset": Netherlands Antilles in 1984, a former territory. That is correct for that question, but it is not what a reader of an early-warning report expects, which is the highest figure among current countries in the latest year. A refined formula adds the two conditions:

```
=INDEX(A2:A11029, MATCH(1, INDEX((C2:C11029=2017)*(B2:B11029="Country")*
        (D2:D11029=MAXIFS(D2:D11029, C2:C11029, 2017, B2:B11029, "Country")), 0), 0))
```

MAXIFS finds the highest calorie value among 2017 countries only. The inner INDEX(..., 0) makes Excel evaluate the three conditions as arrays without Ctrl+Shift+Enter. MATCH(1, ...) finds the first row where all conditions are true. The two versions answer different questions:

Version	Question it answers	Result	Calories
As built (Z2)	Highest value anywhere in the dataset	Netherlands Antilles (1984)	3,877
Refined	Highest among countries in 2017	Belgium	3,768

Both were checked by recalculation: the as-built formula returns Netherlands Antilles and the refined formula returns Belgium. The lesson is to scope a lookup to the question it is meant to answer, and to label the result so the scope is clear.

4.5 Structured references and supporting formulas

The table tblFood lets formulas use column names instead of cell addresses. These Week 2 columns still work and support the Week 3 logic:

Column	Formula	Role
Meat_Check	=ROUND(tblFood[[#This Row],[Meat_kg]] - SUM(tblFood[[#This Row],[Bovine_kg]], ... [OtherMeat_kg]), 2)	Tests that the meat total equals its parts.
Meat_Status	=IF(ABS(tblFood[[#This Row],[Meat_Check]])<=0.2, "Pass", "Check")	Turns the difference into Pass or Check.
Pig_Flag	=IF(tblFood[[#This Row],[Pig_kg]]="", "Not reported", "OK")	Labels blank pigmeat values instead of filling them.

Because these are calculated table columns, they fill down automatically when rows are added, and a column name such as [Meat_kg] keeps working even if columns are moved.

5. Automation with VBA

5.1 What the macro does

GenerateActionReport turns a multi-step filtering job into one click. It scans the Clean_Data table, finds every country that was in Severe Deficit in 2017, copies the key figures into a clean report sheet, highlights each row, and tells the user how many it found.

1. **Prepare the report sheet.** If Action_Required does not exist, create it. If it does, clear it, so old results never linger.
2. **Write the headers** (Entity, Year, Calories, Protein) in bold on a grey background.
3. **Find the last data row** of Clean_Data.
4. **Loop through every row.** If the row is from 2017, is a Country, and is graded Severe Deficit, copy its entity, year, calories, and protein to the next free report row, and shade the row light red.
5. **Finish.** Auto-fit the columns, turn screen updating back on, and show a message with the number of crisis zones found.

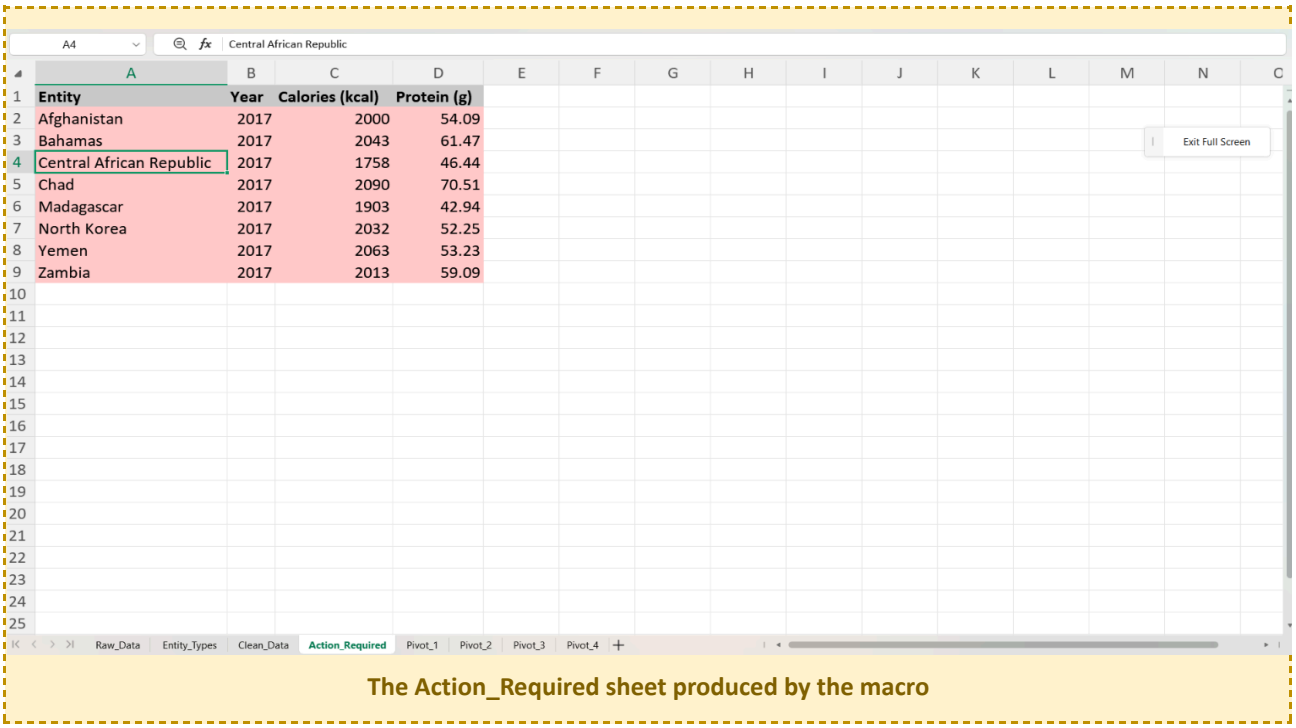
5.2 Code walkthrough

Code	What it does and why
Set wsData = ThisWorkbook.Sheets("Clean_Data")	Points to the source sheet once, so the rest of the code can use the short name wsData.
On Error Resume Next ... On Error GoTo 0	Tries to find Action_Required. If it does not exist, the lookup fails quietly (instead of stopping), and the code then creates the sheet. Error handling is switched straight back on afterwards.
wsReport.Cells.Clear	Wipes the old report so each run starts fresh.
.Value = Array("Entity", "Year", ...)	Writes all four headers in one statement. Font.Bold and Interior.Color style them.
lastRow = wsData.Cells(wsData.Rows.Count, "A").End(xlUp).Row	Starts at the bottom of column A and moves up to the last filled cell, so the loop is the right length, however many rows there are.
If wsData.Cells(i, 3).Value = 2017 And Cells(i, 2).Value = "Country" And Cells(i, 23).Value = "Severe Deficit"	The three-part test. Column 3 is Year, column 2 is Type, and column 23 is Nutritional_Status. All three must be true.
wsReport.Cells(reportRow, 1).Value = wsData.Cells(i, 1).Value (and columns 3, 4, 6)	Copies Entity, Year, Calories, and Protein. reportRow then moves down one line for the next match.
.Interior.Color = RGB(255, 200, 200)	Shades the row light red so urgent rows stand out.
Application.ScreenUpdating = False / True	Stops the screen from redrawing during the loop, which makes the macro faster, and restores it at the end.
MsgBox "... Found " & (reportRow - 2) & " crisis zones."	Reports the count. reportRow starts at 2, so the number of matches is reportRow minus 2.

5.3 The macro's output

Running the macro produced these eight rows on the Action_Required sheet:

#	Entity	Year	Calories (kcal)	Protein (g)
1	Afghanistan	2017	2,000	54.09
2	Bahamas	2017	2,043	61.47
3	Central African Republic	2017	1,758	46.44
4	Chad	2017	2,090	70.51
5	Madagascar	2017	1,903	42.94
6	North Korea	2017	2,032	52.25
7	Yemen	2017	2,063	53.23
8	Zambia	2017	2,013	59.09



The Action_Required sheet produced by the macro

Figure 2. The Action_Required report.

5.4 Intended outcomes

- **Less manual work:** one click replaces filtering by year, by type, and by status, then copying and formatting.
- **Accuracy:** the same three conditions are applied every time, and the Type test keeps regions and former states out.
- **Always current:** the sheet is rebuilt from scratch on every run, so it cannot show stale rows.
- **Clear signal:** red shading and the count in the message box draw attention to the countries that need action.

5.5 Ideas to make it more robust

- **Parameters:** the year (2017), the status text, and the 2,100 and 2,500 thresholds are fixed in the code and the formula. Moving them to labelled input cells would let the report be rerun for any year or threshold without editing code.
- **Column positions:** the macro refers to columns by number (2, 3, 4, 6, 23). If a column is inserted, those numbers shift, and the report would be wrong. Using the table column names would avoid this.
- **Error handling:** an error halfway through the loop would leave screen updating switched off. A handler that always switches it back on would be safer.
- **Sorting:** sorting the output from lowest calories upward would put the most severe case, Central African Republic, at the top.

6. Tutorial: Build and Run It Yourself

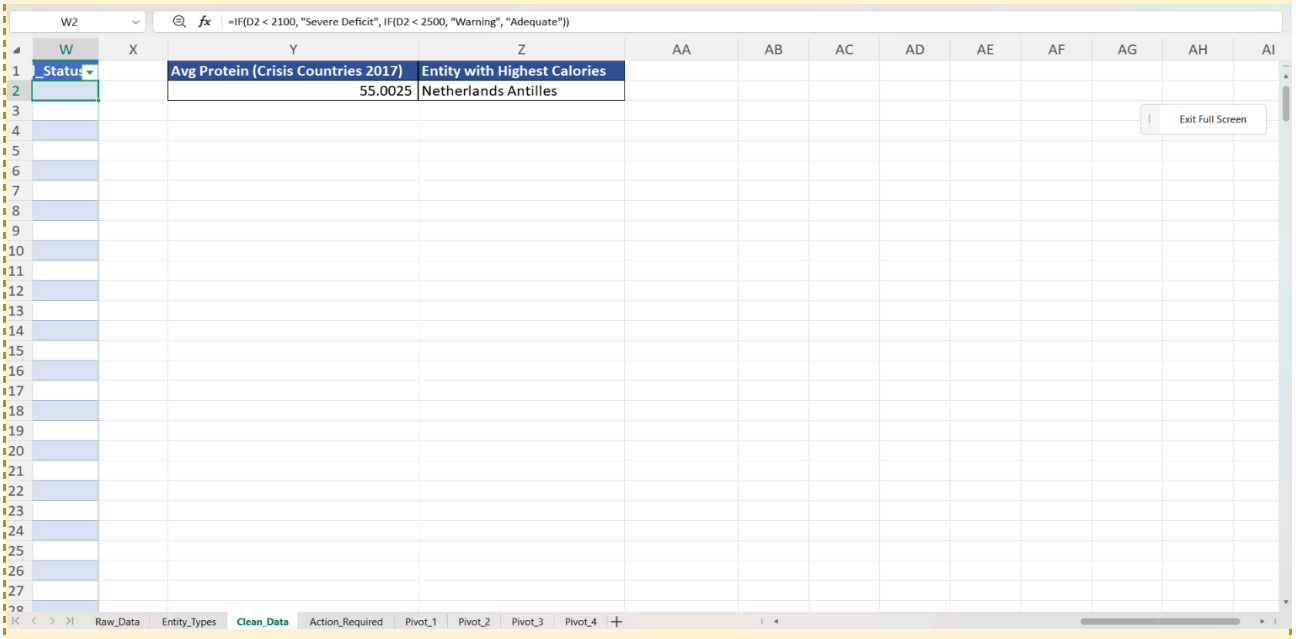
> **Before you start**
>
> **Macros need desktop Excel** (Windows or Mac) and cannot run in Excel for the web. Save the workbook as an **Excel Macro-Enabled Workbook (.xlsm)**, or the macro will be lost.

Part A: Add the Nutritional_Status column

1. Open the Week 2 workbook and go to the **Clean_Data** sheet.
2. In the first empty cell to the right of the table (cell W1), type the header **Nutritional_Status** and press Enter. The table grows by one column.
3. In W2, type: =IF(D2 < 2100, "Severe Deficit", IF(D2 < 2500, "Warning", "Adequate")) and press Enter. Excel fills the formula down all 11,028 rows.
4. Test it: filter the column to Severe Deficit and check that every Calories value shown is below 2,100.

Part B: Add the two result formulas

1. In Y1, type **Avg Protein (Crisis Countries 2017)**. In Y2, enter the SUMPRODUCT formula from Section 4.3.
2. In Z1, type **Entity with Highest Calories**. In Z2, enter the INDEX/MATCH formula from Section 4.4 (or the refined one).
3. Check the values: Y2 should show 55.0025.

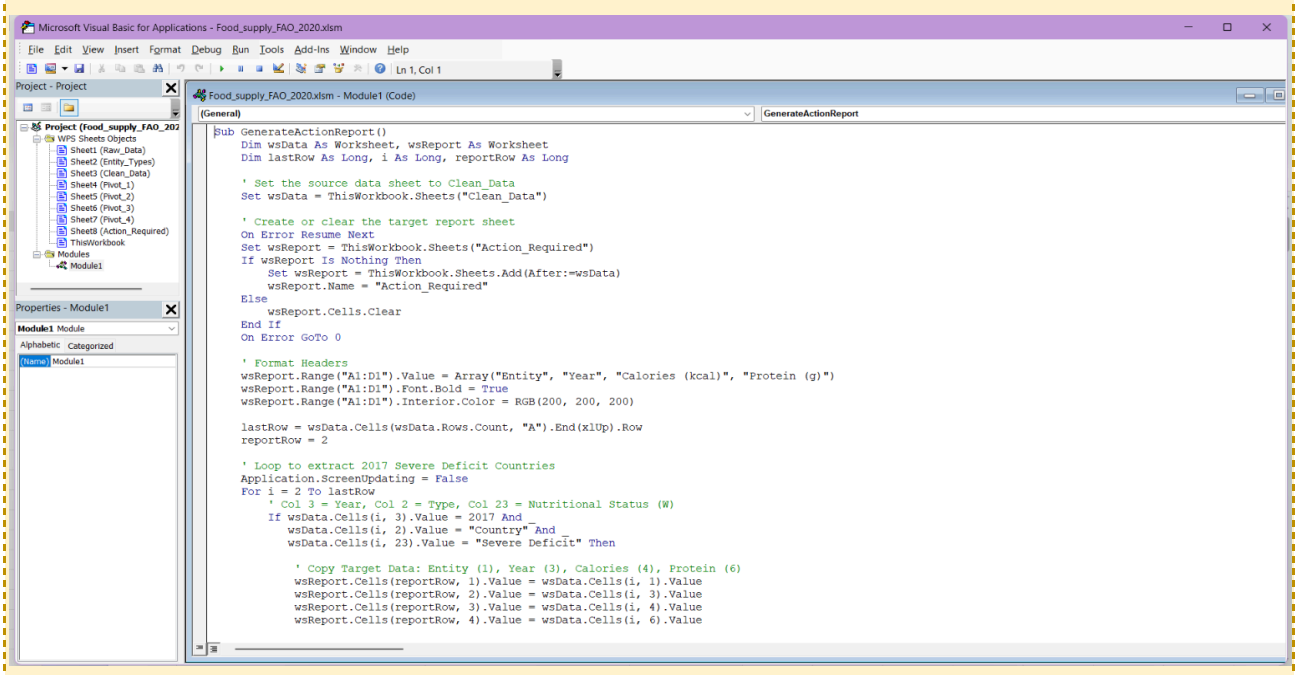


Y1:Z2 with the two results

Figure 3. The two result cells.

Part C: Add the macro

1. Click **File**, then **Options**, then **Customize Ribbon**, tick **Developer**, and click OK.
2. Press **Alt+F11** to open the VBA editor.
3. Click **Insert**, then **Module**. Paste the code from Appendix A into the blank window.
4. Close the editor and click **File**, then **Save As**. Choose **Excel Macro-Enabled Workbook (*.xlsm)**.

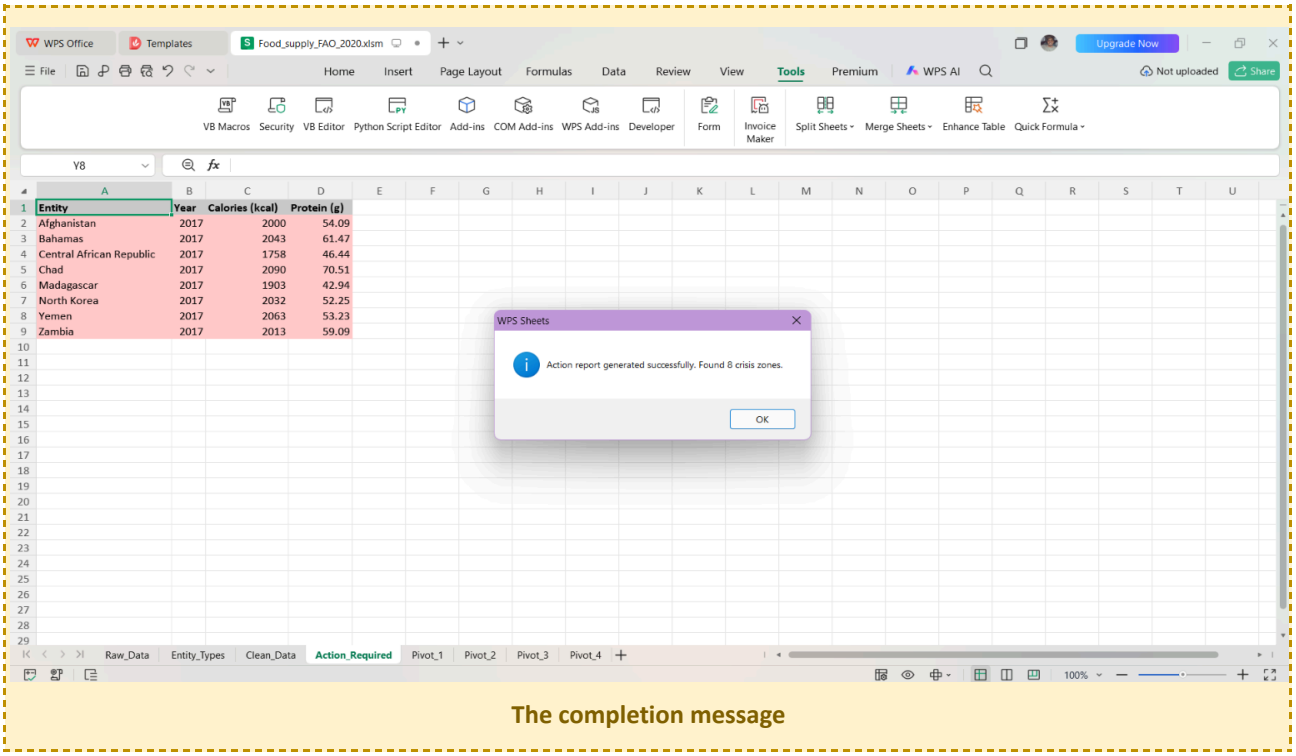


The VBA editor with the macro code in Module1

Figure 4. The macro in the VBA editor.

Part D: Run it

1. Press **Alt+F8**, select **GenerateActionReport**, and click **Run**.
2. Optional: click **Developer**, then **Insert**, then **Button (Form Control)**, draw a button, and assign the macro to it.
3. A message appears: "Action report generated successfully. Found 8 crisis zones."



The completion message

Figure 5. The macro's message.

Part E: Read the result

1. Open the new **Action_Required** sheet: a grey header and eight light-red rows.
2. Run the macro again. Still eight rows (not sixteen), because the sheet is cleared first.

Common problems

Problem	Fix
The macro is greyed out or does nothing	Click Enable Content in the yellow security bar, or allow macros for this file in Trust Center settings.
The report is empty ("Found 0")	Nutritional_Status is not in column W (column 23), or its text differs from "Severe Deficit". The macro matches the text exactly.
"Subscript out of range"	The source sheet is not named Clean_Data.
The macro is missing after reopening	The file was saved as .xlsx. Save it as .xlsm.
The macro is not available at all	You are in Excel for the web. Use desktop Excel.

7. Validation and Testing

7.1 Approach

Four layers of checking protect data reliability. **Boundary tests** check the grading rule at its edges. An **independent recalculation** rebuilds every result a second way and compares. **Reconciliation checks** prove the totals agree with one another. **Macro tests** check that the automation behaves as intended.

7.2 Boundary tests of the grading rule

Boundaries are where grading errors hide, so the formula was tested at and around each threshold, plus three real rows. The formula was recalculated on each input:

Test	Calories	Expected	Actual	Result
One below the first threshold	2,099	Severe Deficit	Severe Deficit	PASS
Exactly on the first threshold	2,100	Warning	Warning	PASS
One below the second threshold	2,499	Warning	Warning	PASS
Exactly on the second threshold	2,500	Adequate	Adequate	PASS
Real row: Central African Republic 2017	1,758	Severe Deficit	Severe Deficit	PASS
Real row: Zimbabwe 2017	2,173	Warning	Warning	PASS

The tests confirm the rule uses "below" (strictly less than): a reading of exactly 2,100 is a Warning, not a Severe Deficit.

7.3 Independent recalculation

- **The Nutritional_Status column.** The grade was recomputed for all **11,028 rows** directly from the calorie figures with the same thresholds, and compared with the grade the workbook shows. **0 mismatches.**
- **The macro output.** The list of 2017 countries below 2,100 kcal was rebuilt separately. It matches the Action_Required sheet exactly: the same **8 countries** with the same calories and protein values, in the same order.
- **The average protein.** The independent average of the eight countries' protein is **55.0025 g**, equal to the value in Y2.

7.4 Reconciliation checks

Check	What it proves	Value A	Value B	Result
R1	Adequate + Warning + Severe Deficit equals the number of rows (all years, all entities)	5,994 + 2,980 + 2,054 = 11,028	11,028	OK
R2	For 2017 countries, the three grades add up to the number of countries	131 + 33 + 8 = 172	172	OK
R3	The macro's row count equals the 2017 Severe Deficit count	8	8	OK

Check	What it proves	Value A	Value B	Result
R4	Every output row is a Country (no regions or former states)	0 non-country rows	0	OK
R5	Y2 equals total protein divided by the count (440.02 / 8)	55.0025	55.0025	OK

7.5 A finding from validation

Checking each result against its label found one cell whose answer did not match what a reader would expect. Cell Z2 ("Entity with Highest Calories") shows Netherlands Antilles, the highest value in the whole dataset (3,877 kcal in 1984, a former territory). Scoped to 2017 countries, the answer is **Belgium (3,768 kcal)**. The formula is not wrong, but its scope was wider than its label suggested. The refined formula in Section 4.4 fixes the scope, and the label should say which version is used.

7.6 Macro tests

Run each test in desktop Excel on a copy of the workbook and complete the yellow **Observed** column. Restore the threshold to 2,100 after tests M4 and M5.

#	Test	Expected result	Expected value	Observed
M1	Run the macro once	Message box and a filled report sheet	8 data rows; "Found 8 crisis zones"	8 rows on Action_Required in the submitted workbook
M2	Run it a second time	The sheet is cleared and rebuilt, not appended to	Still 8 rows	Still 8 rows; previous contents cleared without duplicate rows appended
M3	Delete Action_Required, then run	The sheet is created again	8 rows	Sheet created automatically and populated with 8 rows
M4	Change 2100 to 2200 in the grading formula, then run	More countries are flagged	15 rows	15 rows generated; message box displayed "Found 15 crisis zones"
M5	Change 2100 to 2000, then run	Fewer countries are flagged	2 rows (Central African Republic, Madagascar)	2 rows generated (Central African Republic and Madagascar); "Found 2 crisis zones"
M6	Inspect the output Type	No regions or former states appear	0 non-country rows	0 non-country rows; all 8 records are confirmed Country type

#	Test	Expected result	Expected value	Observed
M7	Check the first and last rows	Same order as the data	Afghanistan first, Zambia last	First row is Afghanistan (2,000 kcal), last row is Zambia (2,013 kcal)

The expected values for M4 and M5 come from the independent calculation: 15 countries are below 2,200 kcal, and 2 are below 2,000 kcal in 2017.

8. Results, Insights and Links to the Week 1 Plan

#	Insight	Evidence	Why it matters
1	**Eight countries are in Severe Deficit in 2017.**	Central African Republic is the lowest at 1,758 kcal, 16.3% below the 2,100 threshold, then Madagascar at 1,903.	These are the first names on the early-warning list.
2	**The picture has improved greatly over time.**	Share of countries in Severe Deficit: 66 of 146 (45.2%) in 1961, 27 of 147 (18.4%) in 1990, 8 of 172 (4.7%) in 2017.	The problem is smaller but concentrated in specific countries.
3	**Crisis countries are short on protein too.**	Average protein is 55.0 g against 81.3 g across all countries in 2017.	Calories and protein fall short together.
4	**A watch list sits just above the line.**	33 more countries are graded Warning. Tajikistan (2,104 kcal) and Kenya (2,124) are closest to the threshold.	Small changes could move them into Severe Deficit.
5	**The threshold changes the list.**	Below 2,000 kcal: 2 countries. Below 2,100: 8. Below 2,200: 15.	A threshold is a decision, so it should be an input and not buried in a formula.

Links to the Week 1 plan

- The macro automates the **Process** and **Package** stations of the FOODFLOW line: it turns the cleaned data into a finished report.
- The Type test (Country only) applies the **consistency gate** from the Week 1 data-quality checks, and the validation steps in Section 7 reuse the idea of independent checks.
- Using table columns, labelled result cells, and a validation section addresses the Week 1 risk of **spreadsheet formula errors**.

9. Limitations

- **Thresholds are assumptions.** The 2,100 and 2,500 kcal rules are screening choices, not official classifications. FAO measures undernourishment by other methods.
- **Supply is not intake.** The data measures food available per person, not food eaten, so a flag is a prompt for investigation and not a diagnosis. Some flagged cases, such as a small import-dependent economy, may reflect how supply is measured.
- **One year only.** The macro is fixed to 2017, the last year in the data.

- **Hard-coded positions and values.** The macro refers to columns by number, and the thresholds sit inside a formula (Section 5.5).
- **Fixed formula ranges.** The formulas in Y2 and Z2 cover rows 2 to 11,029. Added rows would be left out unless the ranges are extended or table references are used.
- **Scope of Z2.** As built, Z2 answers the question for the whole dataset (Section 4.4).
- **Desktop Excel is needed** to run the macro, and the file must be saved as .xlsm.
- **Macro tests need your own run.** Expected values come from an independent calculation, and the Observed column records the outcome of your own runs.

Appendix A: The VBA Macro

This is the code in Module1 of the workbook, exactly as written.

```
Sub GenerateActionReport()
    Dim wsData As Worksheet, wsReport As Worksheet
    Dim lastRow As Long, i As Long, reportRow As Long

    ' Set the source data sheet to Clean_Data
    Set wsData = ThisWorkbook.Sheets("Clean_Data")

    ' Create or clear the target report sheet
    On Error Resume Next
    Set wsReport = ThisWorkbook.Sheets("Action_Required")
    If wsReport Is Nothing Then
        Set wsReport = ThisWorkbook.Sheets.Add(After:=wsData)
        wsReport.Name = "Action_Required"
    Else
        wsReport.Cells.Clear
    End If
    On Error GoTo 0

    ' Format Headers
    wsReport.Range("A1:D1").Value = Array("Entity", "Year", "Calories (kcal)", "Protein (g)")
    wsReport.Range("A1:D1").Font.Bold = True
    wsReport.Range("A1:D1").Interior.Color = RGB(200, 200, 200)

    lastRow = wsData.Cells(wsData.Rows.Count, "A").End(xlUp).Row
    reportRow = 2

    ' Loop to extract 2017 Severe Deficit Countries
    Application.ScreenUpdating = False
    For i = 2 To lastRow
        ' Col 3 = Year, Col 2 = Type, Col 23 = Nutritional Status (W)
        If wsData.Cells(i, 3).Value = 2017 And _
           wsData.Cells(i, 2).Value = "Country" And _
           wsData.Cells(i, 23).Value = "Severe Deficit" Then

            ' Copy Target Data: Entity (1), Year (3), Calories (4), Protein (6)
            wsReport.Cells(reportRow, 1).Value = wsData.Cells(i, 1).Value
            wsReport.Cells(reportRow, 2).Value = wsData.Cells(i, 3).Value
            wsReport.Cells(reportRow, 3).Value = wsData.Cells(i, 4).Value
            wsReport.Cells(reportRow, 4).Value = wsData.Cells(i, 6).Value

            ' Highlight row in red for urgency
            wsReport.Range(wsReport.Cells(reportRow, 1), wsReport.Cells(reportRow, 4)).Interior.Color =
RGB(255, 200, 200)
            reportRow = reportRow + 1
        End If
    Next i

    wsReport.Columns("A:D").AutoFit
    Application.ScreenUpdating = True

    MsgBox "Action report generated successfully. Found " & (reportRow - 2) & " crisis zones.",
vbInformation
End Sub
```